

4.3 Spring Boot Basic Auditing and Advanced Auditing with Hibernate Envers

• Auditing in Spring Boot:

- Auditing in Spring Boot allows you to **automatically** populate certain fields, such as **creation** and **modification timestamps**, as well as the user **who created** or **modified** the entity.

• Steps to Add Auditing:

1. Create **Auditable base Entity** using the following annotations `@EntityListeners(AuditingEntityListener.class)`. This way you will get access to `@CreatedBy`, `@CreatedDate`, `@LastModifiedBy`, and `@LastModifiedDate` annotations.

```
@EntityListeners(AuditingEntityListener.class)
@MappedSuperclass
public abstract class Auditable {

    @CreatedDate
    private Instant createdAt;

    @LastModifiedDate
    private Instant updatedAt;

    @CreatedBy
    private String createdBy;

    @LastModifiedBy
    private String updatedBy;
}
```

- What this means:
 - This is a **common parent class**.
 - All entities will **inherit** these fields.
 - **AuditingEntityListener** tells Spring:
 - "Whenever entity is saved/updated → fill these fields automatically"

2. **Extend** all Entities from the Superclass.

```
@Entity
public class Student extends Auditable {

    @Id
    private Long id;

    private String name;
}
```

- Meaning:
 - Now Student automatically has:
 - createdAt
 - updatedAt
 - createdBy
 - updatedBy
 - No need to write again.

4.3 Spring Boot Basic Auditing and Advanced Auditing with Hibernate Envers

3. Create a class and Implement the **AuditorAware** interface to provide the **current authenticated user** from **Spring Security**.

```
@Component
public class AuditorProvider implements AuditorAware<String> {

    @Override
    public Optional<String> getCurrentAuditor() {
        return Optional.of(SecurityContextHolder
            .getContext()
            .getAuthentication()
            .getName());
    }
}
```

- Spring asks: "Who is the current user?"
- Gets logged-in user.

4. Enable **JPA Auditing** by adding the **@EnableJpaAuditing** annotation to a configuration class.

```
@Configuration
@EnableJpaAuditing
public class JpaConfig {
}
```

- Meaning:
 - You are telling Spring:
 - "Turn ON auditing feature"
 - Without this → nothing works ❌

5. Pass the reference of the **AuditorAware** class bean to the **@EnableJpaAuditing** interface.

- If you have multiple beans, specify:

```
@EnableJpaAuditing(auditorAwareRef = "auditorProvider")
```

- Meaning:
 - Spring needs to know:
 - "Which class gives current user?"
- So you give reference.

• Internal Working of Auditing:

- When you do: `studentRepo.save(student);`
- Behind the scenes:
 - JPA → Hibernate → Entity Lifecycle → Auditing kicks in
- **What is AuditingEntityListener Doing?**
 - I added: `@EntityListeners(AuditingEntityListener.class)`
 - This tells JPA: "Before saving or updating this entity, call this listener."
- **What are @PrePersist and @PreUpdate?**
 - These are JPA lifecycle events.

Event	When it runs
@PrePersist	Before INSERT
@PreUpdate	Before UPDATE

4.3 Spring Boot Basic Auditing and Advanced Auditing with Hibernate Envers

- **Flow for INSERT**

```
studentRepo.save(student);
```

- Internally:

1. Entity is new (**transient** → **persistent**)
2. Hibernate prepares **INSERT**
3. BEFORE insert → **@PrePersist** triggers
4. **AuditingEntityListener** runs
5. Sets:
 - **createdAt**
 - **createdBy**
6. **INSERT** executed

- **Flow for UPDATE**

```
student.setName("New");  
studentRepo.save(student);
```

- Internally:

1. Entity **already exists**
2. Hibernate prepares **UPDATE**
3. BEFORE update → **@PreUpdate** triggers
4. **AuditingEntityListener** runs
5. Sets:
 - **updatedAt**
 - **updatedBy**
6. **UPDATE** executed

- **What Exactly Happens Inside **AuditingEntityListener** ?**

- Simplified logic:

```
@PrePersist  
public void touchForCreate(Object entity) {  
    entity.setCreatedAt(now());  
    entity.setCreatedBy(getCurrentUser());  
}  
  
@PreUpdate  
public void touchForUpdate(Object entity) {  
    entity.setUpdatedAt(now());  
    entity.setUpdatedBy(getCurrentUser());  
}
```

You don't see this code — Spring handles it internally.

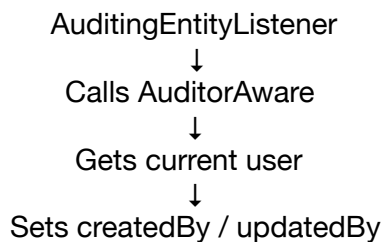
- **Where Does **createdBy** Come From?**

- This is where **AuditorAware** comes in.
- Spring calls: **auditorAware.getCurrentUser()**
- Your implementation:

```
@Override  
public Optional<String> getCurrentAuditor() {  
    return Optional.of("SYSTEM_USER");  
}
```

4.3 Spring Boot Basic Auditing and Advanced Auditing with Hibernate Envers

- **Real Flow**



- Auditing happens **BEFORE SQL** query executes

- **Summary :**

- Spring uses **JPA entity lifecycle callbacks** like **@PrePersist** and **@PreUpdate**.
- **AuditingEntityListener** intercepts these events and sets **audit fields**.
- It uses **AuditorAware** to fetch the **current user**.

- **Hibernate Envers :**

- Hibernate Envers is a module of Hibernate ORM that provides automatic auditing/versioning of your database entities.
- In simple terms:
 - It lets you **track history of your data** — who changed what, and when.
- Think of it like Git for your database.
- Just like Git keeps versions of code, **Envers keeps versions of your entities**.
- Every time you:
 - INSERT
 - UPDATE
 - DELETE
- Envers stores a **snapshot (revision)** of that entity.

- **How It Works:**

1. You add **@Audited** to an entity

```
@Audited
@Entity
public class PostEntity {
    private String title;
}
```

2. Envers creates an audit table automatically

- For: posts
- It creates: posts_AUD

3. Every change creates a revision

- Example:

Revision	Title
1	"Hello"
2	"Hello World"
3	"Final Title"

4.3 Spring Boot Basic Auditing and Advanced Auditing with Hibernate Envers

- **What Data Envers Stores**

Each audit record contains:

- Entity fields (title, description, etc.)
- Revision number
- Revision type:
 - 0 → INSERT
 - 1 → UPDATE
 - 2 → DELETE

- **Example Audit Table (posts_AUD)**

id	title	REV	REVTYPE
1	Hello	1	0
1	Hello World	2	1
1	Final Title	3	1

- **How You Read History**

- Using Envers API:

```
AuditReader reader = AuditReaderFactory.get(entityManager);  
List<Number> revisions = reader.getRevisions(PostEntity.class, 1L);  
PostEntity oldVersion = reader.find(PostEntity.class, 1L, 2);
```

- This gives you:
"What did this entity look like at revision 2?"